

Università Ca' Foscari Venezia
Dipartimento di Scienze Ambientali, Informatica e Statistica

Foundations of Artificial Intelligence

The $N \times N$ Magic Square Problem

Constraint Propagation and Backtracking,
Simulated Annealing, Genetic Algorithms

Student: Wafaa Bajjouk
Student ID: 916145
Professor: Andrea Torsello
Academic year: 2026/2027

7 Nov 2026

Contents

1 Introduction **2**

2 Problem statement **2**

3 Method 1: Constraint propagation and backtracking **3**

3.1 The problem as a constraint satisfaction problem 3

3.2 Constraint propagation 3

3.3 Backtracking 5

3.4 Properties 5

3.5 Experimental setup 5

3.6 Results 6

4 Method 2: Simulated Annealing **8**

4.1 The problem as an optimization problem 8

4.2 Simulated annealing 8

4.3 Properties 9

4.4 Experimental setup 9

4.5 Results 10

5 Method 3: Genetic Algorithms **11**

5.1 Encoding and fitness 11

5.2 Genetic operators 11

5.3 Properties 12

5.4 Experimental setup 12

5.5 Results 13

6 Comparison of the three methods **14**

6.1 Experimental setup 14

6.2 Results 15

6.3 Correctness checks 16

7 Conclusions **16**

1 Introduction

The goal is to fill an $N \times N$ grid with the numbers $1, \ldots, N^2$ so that every row, column and diagonal has the same sum M, possibly with some cells already given. There are $(N^2)!$ ways to fill the grid, so I use three search methods from the course:

- **Backtracking with constraint propagation:** fill one cell at a time and remove the values that can no longer fit. **Result: it solves every instance up to $N = 6$, some up to $N = 8$ with givens, and none from $N = 10$.**
- **Simulated annealing:** start from a full grid and swap two cells at a time, sometimes accepting a worse grid. **Result: without givens it solves every run up to $N = 10$, then 67% at $N = 12$, 22% at $N = 15$ and none at $N = 20$.**
- **Genetic algorithm:** evolve a population of full grids by selection, crossover and mutation. **Result: it solves every run only up to $N = 4$, and none from $N = 6$.** I also tried the PyGAD library, but kept my own code (Section 5).

Each method is described (theory and pseudocode), tested on one 4×4 puzzle, and then run on growing sizes, with and without given cells, with a limit of 10 s per run. **I ran SA and the GA up to $N = 20$ and backtracking up to $N = 15$** ($N = 3, \ldots, 8$, 10, 12, 15, 20). **Every method fails on the largest squares:** backtracking from $N = 10$, the GA from $N = 6$, and SA solves no run at $N = 20$. The tables give the raw number of solved runs, so the failures are visible. The code and the raw results are in the repository `https://github.com/WafaaBajjouk/Foundation_AI_assignments`.

2 Problem statement

A magic square of order N contains each of $1, \ldots, N^2$ once, and every row, column and main diagonal has the same sum M. The numbers add up to $N^2(N^2+1)/2$, shared equally by the N rows, so

$$M = \frac{N(N^2+1)}{2} \qquad (M = 15 \text{ for } N = 3,\ M = 34 \text{ for } N = 4).$$

Let x_{ij} be the value of cell (i, j), $\mathcal{L}$ the set of the $2N+2$ lines (rows, columns, two diagonals), and G the set of given cells with values g_{ij}. I look for x_{ij} such that

$$x_{ij} \in \{1, \ldots, N^2\}, \qquad x_{ij} \neq x_{kl} \ \ \forall (i,j) \neq (k,l),$$
$$\textstyle\sum_{(i,j) \in L} x_{ij} = M \ \ \forall L \in \mathcal{L}, \qquad x_{ij} = g_{ij} \ \ \forall (i,j) \in G.$$

Trying all $(N^2)!$ fillings works for $N = 3$ ($9! = 362\,880$), but not beyond: already $25! \approx 1.6 \cdot 10^{25}$ for $N = 5$.

3 Method 1: Constraint propagation and backtracking

3.1 The problem as a constraint satisfaction problem

The goal state itself is the solution, so the problem is a constraint satisfaction problem [1, 2]:

- **variables:** the N^2 cells x_{ij};
- **domains:** $\{g_{ij}\}$ for a given cell, $\{1, \ldots, N^2\}$ otherwise;
- **constraints:** all values different, every line sums to M.

It is solved by combining propagation (removing impossible values) and search (trying values).

3.2 Constraint propagation

Propagation deletes values that cannot belong to any solution. An arc (v_i, v_j) is arc consistent if

$$\forall x \in D_i \;\; \exists y \in D_j \text{ such that } (x, y) \text{ satisfies the constraint;}$$

values of D_i without such a y are deleted.

All different. If $D_a = \{v\}$, then v is deleted from every other domain.

Line sums. Let a line have k free cells that must sum to R, and let U be the values still possible in them. A value v is deleted if

$$R - v < lo(v) \quad \text{or} \quad R - v > hi(v),$$

where $lo(v)$ and $hi(v)$ are the sums of the $k-1$ smallest and largest values of $U \setminus \{v\}$. If $k = 1$, the last cell must be R.

How much to propagate. After a value is assigned:

- **Forward checking (FC):** revise once only the constraints of the assigned cell. Fast, but removes few values.
- **Full propagation (AC):** repeat the deletions on all constraints until no domain changes. For $x_a \neq x_b$ this is arc consistency (weaker than a global all-different); for the line sums only the bound test is used. Slower per step, but removes more.

Algorithm 1 Constraint propagation

```
1: function ForwardCheck(D, c, v)
2:     delete v from every D_x, x ≠ c
3:     if some D_x = ∅ then
4:         return failure
5:     end if
6:     for each line L containing c do
7:         if ReviseLine(D, L) = failure then
8:             return failure
9:         end if
10:    end for
11:    return ok
12: end function

13: function Propagate(D)                                   ▷ full propagation (AC)
14:    repeat
15:        s ← Σ_x |D_x|
16:        if two cells are fixed to the same value then
17:            return failure
18:        end if
19:        delete the values of the fixed cells from all other domains
20:        if some D_x = ∅ then
21:            return failure
22:        end if
23:        for each line L ∈ 𝓛 do
24:            if ReviseLine(D, L) = failure then
25:                return failure
26:            end if
27:        end for
28:    until Σ_x |D_x| = s
29:    return ok
30: end function

31: function ReviseLine(D, L)
32:    F ← free cells of L;  k ← |F|;  R ← M − sum of the fixed cells of L
33:    if k = 0 then
34:        return ok if R = 0, else failure
35:    end if
36:    if k = 1 then
37:        D_x ← D_x ∩ {R} for the free cell x
38:    else
39:        U ← ⋃_{x∈F} D_x
40:        if |U| < k then
41:            return failure
42:        end if
43:        for each x ∈ F and each v ∈ D_x do
44:            if R − v < lo(v) or R − v > hi(v) then
45:                D_x ← D_x \ {v}
46:            end if
47:        end for
48:    end if
49:    return failure if some D_x = ∅, x ∈ F, else ok
50: end function
```

3.3 Backtracking

Backtracking is a depth-first search: a cell gets a value, the effect is propagated, and the value is undone if a domain becomes empty. The order is chosen during the search:

- **most constrained variable:** the variable with the fewest legal values;
- **least constraining value (LCV):** the value that removes the fewest values from the other domains (the baseline is ascending order).

Algorithm 2 Backtracking with constraint propagation

```
1: function BacktrackingSearch(G, g)
2:     D_c ← {g_c} if c ∈ G, else D_c ← {1, ..., N^2}
3:     if Propagate(D) = failure then
4:         return failure
5:     end if
6:     return Backtrack(D)
7: end function

8: function Backtrack(D)
9:     if |D_c| = 1 for every cell c then
10:        return D if all values differ and all lines sum to M, else failure
11:    end if
12:    c ← cell with |D_c| > 1 and smallest |D_c|
13:    for each v ∈ D_c, in LCV or ascending order do
14:        D' ← copy of D;  D'_c ← {v}
15:        if ForwardCheck(D', c, v) (FC) or Propagate(D') (AC) ≠ failure then
16:            result ← Backtrack(D')
17:            if result ≠ failure then
18:                return result
19:            end if
20:        end if
21:    end for
22:    return failure
23: end function
```

3.4 Properties

- **Complete:** the tree is finite (depth $d = N^2$), so without a time limit it finds a solution if one exists, or proves there is none.
- **Time:** the tree has $O(b^d)$ nodes (b = values tried per cell), and each node also costs a propagation: the time grows exponentially with the number of cells.
- **Memory:** each of the N^2 levels keeps a copy of the N^2 domains of up to N^2 values: $O(N^6)$ in the worst case.
- **Propagation** never removes a solution; it only reduces b.

Expected: AC explores fewer nodes than FC; the time grows exponentially with N; givens shrink the domains from the start, which should help.

3.5 Experimental setup

Instances. I take a known magic square of order N (Siamese method for odd N, standard constructions for even N), rotate or mirror it at random, and keep $\lfloor 0.2N^2 \rfloor$ random cells as givens, so the givens always have a solution. All puzzles of one size come from the same square, which limits their variety.

Experiment 1a. One 4×4 puzzle with three givens ($x_{11} = 13$, $x_{13} = 12$, $x_{24} = 15$), solved

with the four settings (FC or AC, ascending or LCV). I record nodes, backtracks and time, and check that each square is magic and keeps the givens.

Experiment 1b. The four settings for $N = 3, \ldots, 8$, 10, 12, 15, with no givens or 20% given, 5 puzzles each (360 runs).

- **Time limit:** 10 s per run, also checked inside LCV, which is slow on large squares. A failed run keeps its real time (10.000–10.050 s), so the median times include failures.
- **No givens:** the puzzle is the empty grid and the search is deterministic, so the 5 runs differ only in time.
- **Why up to $N = 15$:** at $N = 15$ without givens AC with LCV does not finish a single node in 10 s, so $N = 20$ was not run.
- **Machine:** M1, 16 GB RAM, Python 3.

Code and data. Folder `constraint_propagation_backtracking/`:

- notebook: `constraint_propagation_backtracking.ipynb`;
- raw results: `results_alg1.csv`.

3.6 Results

Propagation	Value order	Nodes	Backtracks	Time (s)
FC	ascending	2286	2280	0.033
FC	LCV	761	755	0.018
AC	ascending	100	97	0.013
AC	LCV	37	34	0.013

Table 1: Experiment 1a. Every run returns a valid magic square that keeps the givens; the four settings do not all return the same square.

```
[21]: for mode in ["fc", "ac"]:
          for order in ["ascending", "lcv"]:
              r = backtracking_search(puzzle, mode=mode, value_order=order)
              ok = is_magic(r["square"]) and \
                   bool(np.all((puzzle == 0) | (puzzle == r["square"])))
              print(f"\n{mode} + {order}: {r['status']}, {r['time']:.3f} s, "
                    f"{r['nodes']} nodes, {r['backtracks']} backtracks, valid: {ok}")
              print(r["square"])

fc + ascending: solved, 0.033 s, 2286 nodes, 2280 backtracks, valid: True
[[13  3 12  6]
 [ 8 10  1 15]
 [ 2 16  7  9]
 [11  5 14  4]]

fc + lcv: solved, 0.018 s, 761 nodes, 755 backtracks, valid: True
[[13  3 12  6]
 [ 8 10  1 15]
 [ 2 16  7  9]
 [11  5 14  4]]

ac + ascending: solved, 0.013 s, 100 nodes, 97 backtracks, valid: True
[[13  3 12  6]
 [10  1  8 15]
 [ 7 16  9  2]
 [ 4 14  5 11]]

ac + lcv: solved, 0.013 s, 37 nodes, 34 backtracks, valid: True
[[13  5 12  4]
 [ 2 11  6 15]
 [ 3  8  9 14]
 [16 10  7  1]]
```

Figure 1: Notebook output of Experiment 1a.

N	3	4	5	6	7	8	10	12	15	20
solved runs (of 5), no givens										
FC + ascending	5/5	5/5	0/5	0/5	0/5	0/5	0/5	0/5	0/5	–
FC + LCV	5/5	5/5	0/5	0/5	0/5	0/5	0/5	0/5	0/5	–
AC + ascending	5/5	5/5	5/5	0/5	0/5	0/5	0/5	0/5	0/5	–
AC + LCV	5/5	5/5	5/5	5/5	0/5	0/5	0/5	0/5	0/5	–
solved runs (of 5), 20% given										
FC + ascending	5/5	5/5	2/5	0/5	0/5	0/5	0/5	0/5	0/5	–
FC + LCV	5/5	5/5	5/5	0/5	0/5	0/5	0/5	0/5	0/5	–
AC + ascending	5/5	5/5	5/5	4/5	0/5	0/5	0/5	0/5	0/5	–
AC + LCV	5/5	5/5	5/5	5/5	3/5	2/5	0/5	0/5	0/5	–
median time (s), no givens										
FC + ascending	0.001	0.004	≥10	≥10	≥10	≥10	≥10	≥10	≥10	–
FC + LCV	0.003	0.025	≥10	≥10	≥10	≥10	≥10	≥10	≥10	–
AC + ascending	0.000	0.003	0.32	≥10	≥10	≥10	≥10	≥10	≥10	–
AC + LCV	0.006	0.063	0.22	0.48	≥10	≥10	≥10	≥10	≥10	–
median time (s), 20% given										
FC + ascending	0.000	0.002	≥10	≥10	≥10	≥10	≥10	≥10	≥10	–
FC + LCV	0.000	0.002	3.78	≥10	≥10	≥10	≥10	≥10	≥10	–
AC + ascending	0.000	0.001	0.38	0.20	≥10	≥10	≥10	≥10	≥10	–
AC + LCV	0.001	0.005	1.30	3.12	7.93	≥10	≥10	≥10	≥10	–

Table 2: Experiment 1b. A run that is not solved failed: it reached the 10 s limit (≥10: the median run failed). –: not run. Raw data: `results_alg1.csv`.

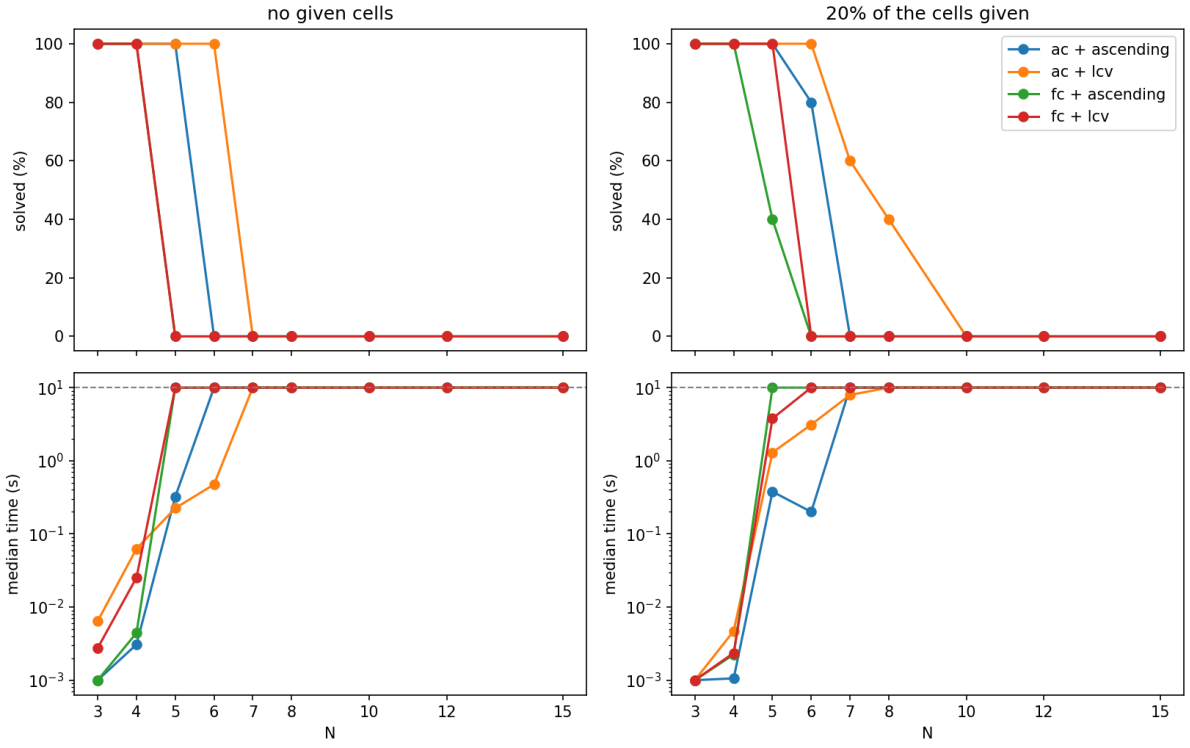


Figure 2: Experiment 1b: solved instances and median time (log scale); dashed line: 10 s limit. In the top-left panel two curves overlap (FC + ascending, FC + LCV).

Analysis.

- **Best setting: AC with LCV**, which solves **every instance up to** $N = 6$, and with givens also 60% at $N = 7$ and 40% at $N = 8$.
- **Propagation and ordering:** at $N = 5$ without givens AC with LCV needs **394 nodes**, while FC with ascending order explores **about 1.2 million** in 10 s without a solution.
- **Cost of LCV:** one propagation per candidate value, so it is slower on small squares (0.063 s against 0.003 s at $N = 4$) and pays off from $N = 5$.
- **Givens** help at $N = 7, 8$ (60% and 40% against 0%), but slow the search down at $N = 5, 6$.
- **As N grows: from $N = 10$ no setting solves anything**. Each node costs more (N^2 domains of up to N^2 values), so AC with LCV finishes only 10 nodes at $N = 10$ and none at $N = 15$ (without givens); FC nodes are cheaper but it prunes too little.

Method 1 is exact, but within 10 s it is practical only up to about $N = 6$–8. AC with LCV is used in the comparison of Section 6.

4 Method 2: Simulated Annealing

4.1 The problem as an optimization problem

Local search turns the CSP into an optimization problem [1]: minimize a function that is 0 only when all constraints hold.

- **state:** a full grid with each of $1, \ldots, N^2$ once and the givens in place, so all values always differ;
- **objective (energy):**
$$f(s) = \sum_{L \in \mathcal{L}} \Big| \sum_{(i,j) \in L} x_{ij} - M \Big|,$$
which is 0 if and only if s is a magic square;
- **neighbour:** swap the values of two free cells.

4.2 Simulated annealing

Hill climbing stops in local minima. Simulated annealing picks a random neighbour s^* and, with $\Delta = f(s^*) - f(s)$:

- **if $\Delta < 0$:** always moves to s^*;
- **otherwise:** moves to s^* with probability $e^{-\Delta/T}$.

The temperature T starts high (worse moves often accepted) and decreases (at the end almost only better moves).

- **schedule:** T_0 = mean increase of f over the worsening swaps among 200 random ones; then $T \leftarrow \alpha T$ at each step, down to 0.01 after $L = 2000N^2$ steps;
- **restart:** if a run ends with $f(s) > 0$, start again from a new random state, until the time limit.

Algorithm 3 Simulated annealing

```
1: function SimulatedAnnealing(G, g)
2:     repeat
3:         s ← givens in place, the other values shuffled into the free cells
4:         T ← T_0;  α ← (0.01/T_0)^{1/L}
5:         for k = 1, ..., L do
6:             if f(s) = 0 then
7:                 return s
8:             end if
9:             s* ← s with two random free cells swapped
10:            Δ ← f(s*) − f(s)
11:            if Δ < 0 or random() < e^{−Δ/T} then
12:                s ← s*
13:            end if
14:            T ← αT
15:        end for
16:    until time limit reached
17:    return failure
18: end function
```

4.3 Properties

- **Not complete:** it can miss a solution and cannot prove that the givens have none.
- **Convergence:** if T decreases slowly enough, it reaches the global optimum with probability approaching 1 [1, 3].
- **Cost per step:** the line sums are stored and updated after each accepted swap; a swap changes at most 8 lines, so Δ costs constant time. Memory $O(N^2)$.

Expected: SA reaches larger N than backtracking; some runs end in a local minimum, so restarts are used; givens may make it harder, since SA cannot use them to prune.

4.4 Experimental setup

Instances. Built as for Method 1.

Experiment 2a. The 4×4 puzzle of Experiment 1a, solved once; I record the energy during the search.

Experiment 2b. $N = 3, \ldots, 8$, 10, 12, 15, 20, with no givens or 20% given, 3 puzzles and 3 runs per puzzle (180 runs).

- **Seeds:** separate seeds for the puzzle and the run, so the difficulty of the puzzle and the randomness of SA are not mixed.
- **Time limit:** 10 s per run, checked every 1000 steps; failed runs keep their real time, as for Method 1.
- **Recorded:** solved runs, median time and median number of restarts.
- **Why up to $N = 20$:** a swap costs constant time, so SA can try large squares; 20 goes beyond 15 to see where it stops.

Code and data. Folder `simulated_annealing/`:

- notebook: `simulated_annealing.ipynb`;
- raw results: `results_sa.csv`.

4.5 Results

Experiment 2a. The 4×4 instance is solved in 0.044 s and 6181 iterations, without restarts. The square found keeps the givens and is the same one found by Method 1 with AC and ascending order.

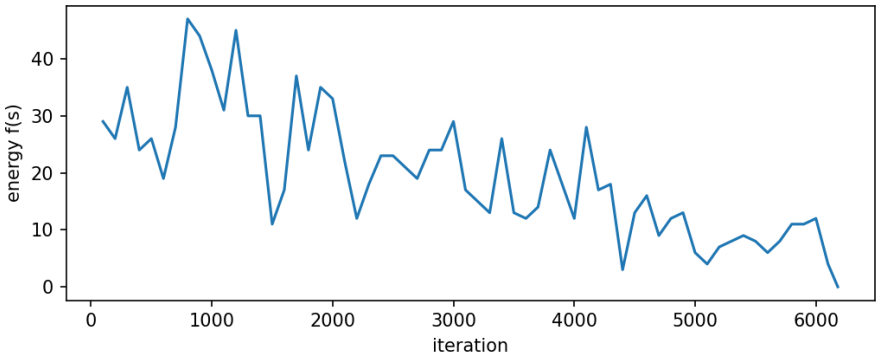


Figure 3: Experiment 2a: energy during the search. At high temperature worse states are often accepted; as T decreases the energy falls to 0.

Experiment 2b.

N	3	4	5	6	7	8	10	12	15	20
solved runs (of 9)										
no givens	9/9	9/9	9/9	9/9	9/9	9/9	9/9	6/9	2/9	0/9
20% given	9/9	9/9	8/9	6/9	6/9	4/9	5/9	1/9	1/9	0/9
median time (s)										
no givens	0.004	0.04	1.39	1.49	1.31	1.42	1.97	7.39	≥10	≥10
20% given	0.002	0.03	3.04	8.76	6.00	≥10	7.07	≥10	≥10	≥10
median restarts										
no givens	0	0	7	5	3	2	2	6	6	3
20% given	0	0	16	33	16	21	9	9	6	3

Table 3: Experiment 2b (9 runs per point). A run that is not solved failed: it reached the 10 s limit (≥10: the median run failed). Raw data: `results_sa.csv`.

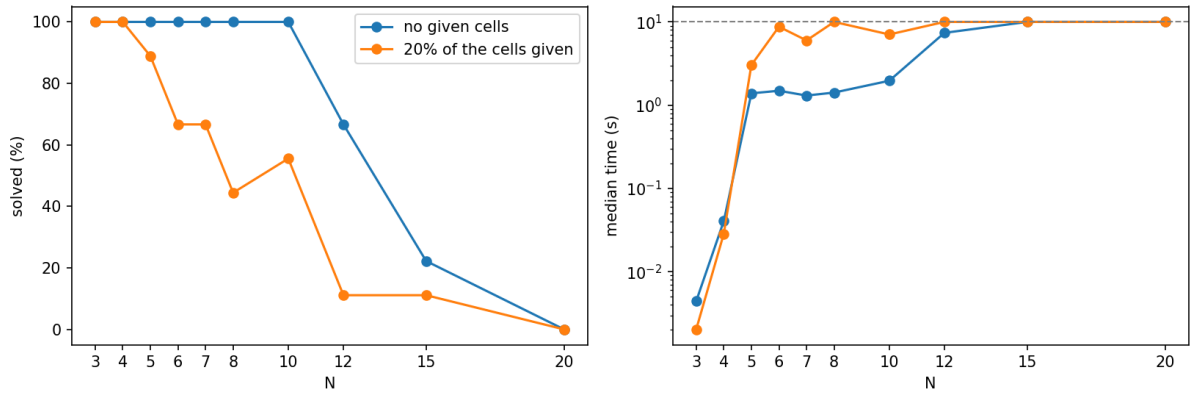


Figure 4: Experiment 2b: solved instances and median time (log scale); dashed line: 10 s limit.

Analysis.

- **Without givens: every run solved up to** $N = 10$, with median times below 2 s; still 6/9 at $N = 12$ and 2/9 at $N = 15$, and **none at** $N = 20$.
- **With givens:** weaker: 44–67% for $N = 6$–10, 11% at $N = 12, 15$.
- **Restarts:** from $N = 5$ most runs need several (median 2–7 without givens, 3–33 with): one run often ends in a local minimum.
- **As** N **grows:** one run takes $2000N^2$ steps (800,000 at $N = 20$), so only a few restarts fit in 10 s (median 3 at $N = 20$): beyond $N = 10$ the time limit matters as much as the method.
- With 9 runs per point, one run changes the rate by 11%.

SA is the method that scales furthest: every run up to $N = 10$ **without givens.**

5 Method 3: Genetic Algorithms

5.1 Encoding and fitness

A genetic algorithm [1, 4] keeps a population of k states (strings) and builds new ones by combining two parents; a fitness function gives higher values to better states.

- **encoding:** the values of the free cells, row by row (the givens are fixed and not in the string);
- **fitness:** $\phi(s) = 1/(1 + f(s))$ with f the energy of Method 2, so $\phi(s) = 1$ only for a magic square.

5.2 Genetic operators

Each generation is built by:

- **selection (roulette wheel):** individual i enters the mating pool with probability $p_i = \phi_i / \sum_k \phi_k$;
- **crossover:** with probability p_c, cut two parents at a random point and swap the right parts; a repeated value is then replaced by a missing one (repair);
- **mutation:** in each child, with probability p_m, swap two positions, so the string stays a permutation;
- **restart:** if the best energy does not improve for 300 generations, start from a new random population.

I use $k = 200$, $p_c = 0.9$ and $p_m = 0.3$.

Library version. I also tried the PyGAD library for selection, crossover and the generation loop. Its handling of repeated values does not fit a permutation encoding, and its swap mutation always uses the same pair of positions, so my repair and mutation were still needed. I therefore kept my own implementation; the library test is not part of the reported experiments.

Algorithm 4 Genetic algorithm

```
function GeneticAlgorithm(G, g)
    P ← k random strings (the free values shuffled)
    repeat
        if some s ∈ P has f(s) = 0 then
            return s
        end if
        if no improvement for 300 generations then
            P ← k new random strings
        end if
        pool ← k individuals drawn from P with probability p_i
        for each pair (p_1, p_2) in pool do
            c_1 ← p_1, c_2 ← p_2
            with probability p_c: (c_1, c_2) ← Crossover(p_1, p_2), then repair
            for each child c ∈ {c_1, c_2}, with probability p_m: swap two positions in c
        end for
        P ← the k children
    until time limit reached
    return failure
end function
```

5.3 Properties

- **Not complete:** like SA, it cannot prove that the givens have no solution.
- **Cost per generation:** the $2N + 2$ line sums of k individuals, $O(kN^2)$ in principle; my code uses a dense matrix product, which costs $O(kN^3)$. Memory $O(kN^2 + N^3)$: the population and the dense line matrix.
- **Weak selection:** with $\phi = 1/(1 + f)$, grids with close energies get close selection probabilities, so good grids are favoured only a little.
- **No elitism:** without elitism, the best grid can be lost between generations.
- **Crossover on permutations:** the repair changes further values, so good lines of the parents are often broken.

Expected: long periods without improvement, so restarts are needed; the GA solves only small squares.

5.4 Experimental setup

Instances. Built as for Method 1.

Experiment 3a. The 4×4 puzzle of Experiment 1a, solved once; I record the best energy in the population at each generation.

Experiment 3b. The same sizes, puzzles and runs as Experiment 2b: $N = 3, \ldots, 8$, 10, 12, 15, 20, with no givens or 20% given, 3 puzzles and 3 runs per puzzle (180 runs).

- **Time limit:** 10 s per run; failed runs keep their real time.
- **Recorded:** solved runs, median time and median best energy reached.
- **Why up to $N = 20$:** one generation costs $O(kN^3)$, but the large sizes are kept, as for SA, to see where the GA stops.

Code and data. Folder `genetic_algorithm/`:

- notebook: `genetic_algorithm.ipynb`;
- raw results: `results_ga.csv`.

5.5 Results

Experiment 3a. The 4×4 instance is solved in 2.24 s, after 1425 generations and 4 restarts; the square is the same one found by Method 1 with AC and LCV. On the same instance SA needs 0.04 s.

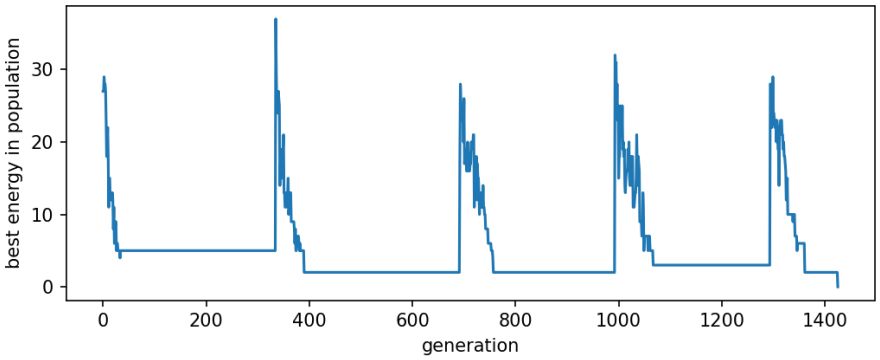


Figure 5: Experiment 3a: best energy in the population at each generation. The flat parts are plateaus where the population stops improving; the jumps are restarts.

Experiment 3b.

N	3	4	5	6	7	8	10	12	15	20
solved runs (of 9)										
no givens	9/9	9/9	2/9	0/9	0/9	0/9	0/9	0/9	0/9	0/9
20% given	9/9	9/9	0/9	0/9	0/9	0/9	0/9	0/9	0/9	0/9
median time (s)										
no givens	0.03	1.17	≥10	≥10	≥10	≥10	≥10	≥10	≥10	≥10
20% given	0.011	0.72	≥10	≥10	≥10	≥10	≥10	≥10	≥10	≥10
median best energy										
no givens	0	0	2	3	63	150	431	962	2386	7712
20% given	0	0	3	4	8	101	391	838	2226	6699
median restarts										
no givens	0	2	15	12	9	8	6	3	3	1
20% given	0	1	16	13	10	9	7	4	3	0

Table 4: Experiment 3b (9 runs per point). A run that is not solved failed: it reached the 10 s limit (≥10: the median run failed). Raw data: `results_ga.csv`.

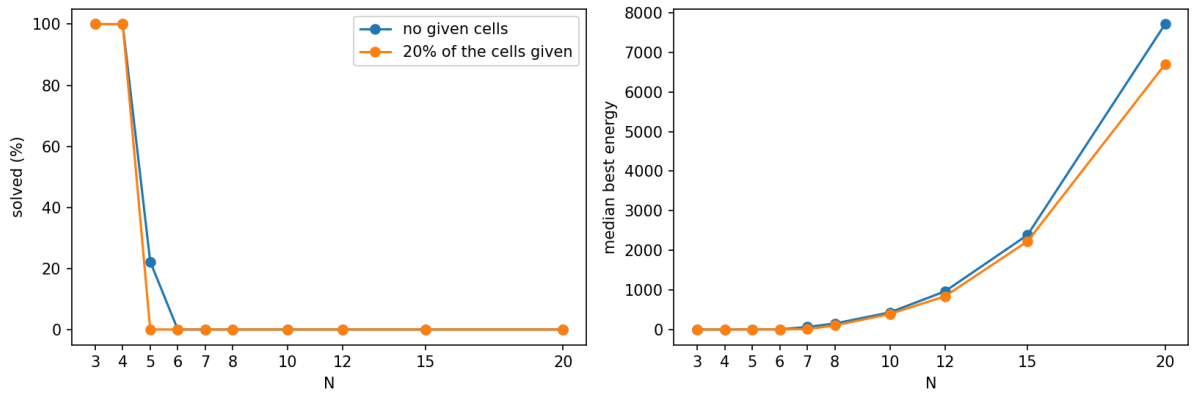


Figure 6: Experiment 3b: solved runs and median best energy reached.

Analysis.

- **Solved: every run only up to** $N = 4$; 2 of 9 at $N = 5$ without givens; **nothing from** $N = 6$.
- **As** N **grows:** the best energy is small at $N = 5, 6$ (2–4), but grows fast (63 at $N = 7$, about 7,700 at $N = 20$): on large squares the GA ends far from a solution.
- **Restarts:** frequent at $N = 5$–8 (median 8–16), fewer on large squares (1 or 0 at $N = 20$), since a generation costs $O(kN^3)$ and few fit in 10 s.
- **Speed:** even when it succeeds it is slow: about 50 times slower than SA on the 4×4 puzzle.

The GA is the weakest method. The properties of Section 5.3 give a possible explanation: selection favours the best grids only a little, the best grid can be lost between generations, and crossover breaks correct lines. This matches the long plateaus of Figure 5, but I did not test it directly.

6 Comparison of the three methods

6.1 Experimental setup

No new runs. I compare the results of Experiments 1b, 2b and 3b on the puzzles all three share.

- **Sizes:** $N = 3, \ldots, 8$, 10, 12, 15, 20 (backtracking up to 15), with no givens or 20% given.
- **Puzzles:** the same 3 per point (seeds $0, 1, 2$), the same 10 s limit and the same machine.
- **Backtracking:** AC with LCV, its best setting; one run per puzzle, since it is deterministic. SA and the GA have three.
- **Recorded:** solved runs out of the total and the median time, computed by `comparison/comparison.ipynb`.

These are 3 of the 5 puzzles of Experiment 1b, so the rates can differ from Table 2: at $N = 7$ with givens backtracking solves all 3 shared puzzles, but 60% of all 5.

6.2 Results

N	3	4	5	6	7	8	10	12	15	20
Backtracking (AC + LCV), 1 run per puzzle										
no givens	3/3	3/3	3/3	3/3	0/3	0/3	0/3	0/3	0/3	–
20% given	3/3	3/3	3/3	3/3	3/3	1/3	0/3	0/3	0/3	–
Simulated annealing, 3 runs per puzzle										
no givens	9/9	9/9	9/9	9/9	9/9	9/9	9/9	6/9	2/9	0/9
20% given	9/9	9/9	8/9	6/9	6/9	4/9	5/9	1/9	1/9	0/9
Genetic algorithm, 3 runs per puzzle										
no givens	9/9	9/9	2/9	0/9	0/9	0/9	0/9	0/9	0/9	0/9
20% given	9/9	9/9	0/9	0/9	0/9	0/9	0/9	0/9	0/9	0/9

Table 5: Solved runs on the shared instances (3 puzzles per point; –: not run).

N	3	4	5	6	7	8	10	12	15	20
Backtracking (AC + LCV)										
no givens	0.007	0.063	0.22	0.48	≥10	≥10	≥10	≥10	≥10	–
20% given	0.001	0.006	1.30	2.67	1.61	≥10	≥10	≥10	≥10	–
Simulated annealing										
no givens	0.004	0.04	1.39	1.49	1.31	1.42	1.97	7.39	≥10	≥10
20% given	0.002	0.03	3.04	8.76	6.00	≥10	7.07	≥10	≥10	≥10
Genetic algorithm										
no givens	0.03	1.17	≥10	≥10	≥10	≥10	≥10	≥10	≥10	≥10
20% given	0.011	0.72	≥10	≥10	≥10	≥10	≥10	≥10	≥10	≥10

Table 6: Median time in seconds on the shared instances (≥10: the median run reached the limit; failed runs take about 10 s).

Analysis.

- **Small squares ($N \leq 4$):** all three solve every instance; backtracking and SA in milliseconds, the GA in about 1 s.
- **No givens: backtracking up to $N = 6$, the GA up to $N = 4$, SA up to $N = 10$;** then SA drops too (6/9 at $N = 12$, 2/9 at $N = 15$, 0/9 at $N = 20$).
- **With givens: backtracking solves all shared puzzles up to $N = 7$;** SA fails some runs from $N = 5$, the GA solves nothing from $N = 5$. SA and the GA keep the givens fixed but cannot use them to prune.
- **As N grows:** from $N = 10$ backtracking and the GA solve nothing, and **at $N = 20$ neither SA nor the GA solves any run** in 10 s (backtracking was tested up to $N = 15$).
- **Time:** without givens backtracking is faster than SA at $N = 5, 6$ (0.22 s against 1.39 s, 0.48 s against 1.49 s), but stops at $N = 7$.
- **Few runs:** 3 puzzles per point, so only large differences are clear, such as SA (9/9) against the GA (0/9) without givens at $N = 6$, 7, 8 and 10.

Theory against evidence. Table 7 puts the properties from the theory next to what the experiments showed.

Property	Backtracking (AC + LCV)	Simulated annealing	Genetic algorithm
Complete	yes; solves every shared puzzle it finishes	no; needs restarts	no; needs restarts
Proves "no solution"	yes; 0.00 s on the impossible 3×3	no; stops at the time limit	no; stops at the time limit
Uses givens to prune	yes; with givens it solves $N = 7, 8$ (60%, 40% against 0%)	no; weaker with givens (44% against 100% at $N = 8$)	no; 0/9 from $N = 5$ with givens
Memory	$O(N^6)$ worst case	$O(N^2)$	$O(kN^2 + N^3)$
Cost per step	one propagation per node, one per value with LCV	constant per swap	$O(kN^3)$ per generation
Largest N with every run solved, no givens	6	10	4

Table 7: Properties from the theory and the evidence from the experiments (Tables 2, 3, 4, 8).

6.3 Correctness checks

The last cells of each notebook test special cases (2 s limit). Every method first rejects givens with a repeated value or a value outside $1, \ldots, N^2$. Every square reported as solved is then checked by an independent function, written without the solver's code: each of $1, \ldots, N^2$ once, every line summing to M, givens kept. Every square reported as solved passed the independent check.

Case	Backtracking	Simulated annealing	Genetic algorithm
repeated given (3×3)	invalid input	invalid input	invalid input
given out of range (3×3)	invalid input	invalid input	invalid input
centre = 1 (3×3, no solution)	no solution, 0.00 s	time limit	time limit
complete 4×4 square	solved	solved	solved
1 or 2 empty cells (4×4)	solved	solved	solved
random puzzles, $N = 3, 4$	20/20	20/20	17/20

Table 8: Correctness checks (2 s limit). Every square reported as solved passed the independent check; the random puzzles are seeds $0, \ldots, 4$ with no givens or 20% given.

As the theory says, only backtracking proves that the impossible puzzle has no solution; SA and the GA cannot tell it from a hard one and stop at the time limit.

7 Conclusions

- **Backtracking** is exact and complete, and **the only method that can prove a puzzle has no solution**. AC with LCV cuts the search a lot (394 nodes at $N = 5$, against about 1.2 million for FC with ascending order), but the time still grows exponentially: **every instance up to $N = 6$, nothing from $N = 10$.**
- **Simulated annealing** is not complete, but **scales furthest: every run up to $N = 10$ without givens**, with median times below 2 s. Cheap swaps and accepting worse moves are plausible reasons. Beyond $N = 10$ it still solves 6/9 at $N = 12$ and 2/9 at $N = 15$, but none at $N = 20$. Givens make it weaker.

- **The genetic algorithm** is **the weakest: every run only up to** $N = 4$. Roulette selection on $\phi = 1/(1 + f)$ favours good grids only a little, without elitism the best grid can be lost between generations, and crossover breaks correct lines.

Why the methods fail on large squares.

- **Backtracking:** the tree grows exponentially, and each node costs more as N grows (N^2 domains of up to N^2 values, one propagation per value with LCV): at $N = 15$ without givens AC with LCV does not finish even one node in 10 s.
- **SA:** one run takes $2000N^2$ steps, so at $N = 20$ only about 3 restarts fit in 10 s (median), which was not enough.
- **GA:** weak selection, no elitism and crossover that breaks correct lines (a possible explanation), and each generation costs $O(kN^3)$.

In short: backtracking is the better choice for small squares, with givens, or to prove there is no solution; simulated annealing for larger squares without givens, up to $N = 10$. Beyond that results get poor: SA solves 6/9 runs at $N = 12$ and 2/9 at $N = 15$, and at $N = 20$ neither SA nor the GA solves any run (backtracking was tested up to $N = 15$). With few runs per point, these results show trends rather than exact values.

References

[1] A. Torsello, *Foundations of Artificial Intelligence*, lecture slides: Blind Search, Constraint Satisfaction, Local Search. Università Ca' Foscari Venezia.

[2] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.

[3] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by simulated annealing, *Science* 220(4598), 671–680, 1983.

[4] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.